

Satellite Ground Control Dashboard

Manual de Instruções

Repositório e Estrutura

O código-fonte do projeto está organizado da seguinte forma:

dashboard/ — Backend (Kotlin + Spring Boot) — expõe REST API e WebSocket na porta 8080.

frontend/ — Frontend (React + Vite + TypeScript) — interface web na porta 5173.

Executar o Backend

Pré-requisitos: Java 21+. O Gradle Wrapper está incluído — não é necessário instalar o Gradle.

Numa consola aberta na diretoria **dashboard/dashboard**:

```
Linux/Mac: ./gradlew bootRun
```

```
Windows: gradlew.bat bootRun
```

O backend fica disponível em **http://localhost:8080**.

Executar o Frontend

Pré-requisitos: Node.js 18+.

Numa consola aberta na diretoria **dashboard/frontend**:

```
npm install
```

```
npm run dev
```

O frontend fica disponível em **http://localhost:5173**. O Vite faz proxy automático de **/api/*** para o backend em :8080.

Ligar ao Satélite via Porta COM

1. Abre o browser em **http://localhost:5173**.
2. No painel **Porta COM**, clica em **■ Actualizar** para listar as portas disponíveis.
3. Selecciona a porta correta (ex: **COM3**) e o *baud rate*.
4. Clica em **■ Ligar**. O dashboard começa a receber dados em tempo real via WebSocket.

Enviar Firmware via OTA

1. Faz o build do firmware no MPLAB X IDE.

2. Localiza o ficheiro **.hex** gerado (ex: **PS_OBC.X.production.hex**).
3. No dashboard, abre o painel **OTA Update**.
4. Arrasta ou selecciona o ficheiro **.hex**.
5. Clica em **■ Enviar Firmware via OTA**. O backend envia os registos Intel HEX linha a linha via UART.

Formato de Dados Esperado (UART)

O backend aceita duas formas de dados enviados pelo satélite:

JSON (telemetria automática):

```
{"temp": 23.4, "volt": 3.7, "curr": 0.15, "bat": 82, "alt": 400.0, "lat": 38.7, "lon": -9.1, "rssi": -87}
```

Texto livre (logs):

Qualquer linha que não seja JSON é tratada como log. Linhas com prefixo **ERR/ERROR** → tipo **ERROR**; **WARN** → **WARNING**; restantes → **RESPONSE**.

PS_OBC.X — ISEL On-Board Computer

Manual de Instruções

Sobre o Projeto

Firmware bare-metal para o computador de bordo (OBC) de um nanossatélite, desenvolvido para o microcontrolador **ATSAMV71Q21** (Cortex-M7 a 300 MHz). A arquitetura assenta em máquinas de estados (FSM) e execução não-bloqueante (super-loop cooperativo), com alocação de memória totalmente estática.

Estrutura do Repositório

src/app/ — Lógica de missão, máquinas de estado, modos de operação e leitura de sensores.

src/hal/ — Hardware Abstraction Layer — inicialização de clocks, GPIO, SysTick, UART de debug.

src/drivers/ — Drivers não-bloqueantes para USART, I2C, SPI e QSPI.

src/peripherals/ — Módulos de alto nível para cada periférico: GNSS, IMU, propulsor, EPS, memória externa, pressão, temperatura e TT&C.;

src/testsForBoard/ — Testes unitários e testes determinísticos para validação em hardware.

inc/ — Ficheiros de cabeçalho (.h) espelhando a estrutura de src/.

ota/ — Suporte ao mecanismo de atualização Over-The-Air.

Pré-requisitos

- **MPLAB X IDE** (v6.x ou superior) — <https://www.microchip.com/mplab/mplab-x-ide>
 - **XC32 Compiler** (v4.x ou superior) — licença gratuita suficiente.
 - **MPLAB Harmony 3** (opcional, apenas para regenerar código de configuração).
 - **ATMEL-ICE** ou **MPLAB PICKit 4** para programar/depurar via JTAG/SWD.
-

Compilar o Projeto

No MPLAB X IDE:

1. Abre o projeto: **File** → **Open Project...** e seleciona a pasta **PS_OBC.X**.
 2. Confirma que o compilador XC32 está configurado em **Project Properties** → **Compiler Toolchain**.
 3. Clica em **Build Main Project** (martelo) ou prime **F11**.
 4. O ficheiro de saída gerado é **dist/default/production/PS_OBC.X.production.hex**.
-

Programar o Microcontrolador

1. Liga o programador (ATMEL-ICE ou PICKit 4) ao conector JTAG/SWD da placa.
 2. Em **Project Properties** → **Connected Hardware Tool**, selecciona o programador.
 3. Clica em **Make and Program Device** (seta para baixo) ou prime **F6**.
 4. O MPLAB X compila (se necessário) e faz o flash automaticamente.
-

Monitor de Debug (UART)

O firmware imprime mensagens de diagnóstico via UART de debug. Para as visualizar, liga um conversor USB-UART ao pino de TX de debug e abre um terminal série:

```
Baud rate: 115200 | 8N1 | sem controlo de fluxo
```

No arranque deverá aparecer: **=== OBC BOOT === FW v2.0**

Atualização OTA (Over-The-Air)

O OBC suporta atualização de firmware em voo via UART (TT&C;). O ficheiro **.hex** é enviado a partir do Ground Control Dashboard, que transmite os registos Intel HEX linha a linha com um delay de 20 ms entre cada um para que o bootloader os processe sem erros.

Consulta o manual do Dashboard para os passos de envio.
